Image classification with Hybrid Quantum Convolutional Neural Networks

QCML2

1st Miguel Castela
DEI, Universidade de Coimbra
uc2022212972@student.uc.pt

2nd Professor Jorge Lobo
ISR, Universidade de Coimbra
jlobo@isr.uc.pt

ABSTRACT

Artificial neural networks are currently one of the most popular models used in machine learning. A subcategory of these called convolutional neural networks (CNN) is particularly popular. These are networks mainly used for the task of image recognition. Using pooling and convolution layers they extract structured information about images. This project will be related to CNN's but also to Quantum computing. Quantum machine learning leverages the principles of quantum mechanics to process information, allowing for the creation of algorithms that can potentially outperform classical counterparts. By harnessing qubits and applying rotations, these algorithms can explore multiple possibilities simultaneously, offering a unique approach to solving complex problems. While still in the early stages of development, quantum machine learning holds promise for revolutionizing certain computational tasks, particularly those involving large-scale optimization and data analysis.

INTRODUCTION AND METHODOLOGY

This project explores the integration of quantum computing with neural network architectures, focusing on two types of quantum neural networks (QNNs) from the Qiskit tutorials. A final neural network was then created with the best features of each of the tutorials and some evolutionary-inspired approaches to the hybrid quantum convolutional neural network (HQCNN), fist proposed by Cong et. al. [[**?**]].

First evaluations with the Qiskit tutorials

First tutorial

The first step of this project was using the Qiskit framework, combined with pythorch to construct a HQNN. After studying the first three qiskit tutorials ("Quantum Neural Networks", "Neural Network Classifier" & "Training a Quantum Model on a Real Dataset") to learn the basics , the tutorial "Torch Connector and Hybrind QNNs" [1] was implemented in totality. This tutorial focuses on the development of a standard HQCNN:

- The first step is to initialize the Estimator. The Estimator is a class in Qiskit that allows you to estimate the expectation values of quantum circuits. It is used to evaluate the performance of quantum models by measuring their outputs.

```
estimator = Estimator()
```

- Then, using the torchvision API [2], the MNIST training dataset is loaded (The MNIST database is a large database of handwritten digits [3]) and a transformation is applied to convert images to tensors.
  A tensor is a generalization of vectors and matrices to any number of dimensions. They are the fundamental data structure in PyTorch, similar to NumPy arrays but with additional capabilities for GPU acceleration, beneficial for this type of work.

```
X_train = datasets.MNIST(
    root="./data", train=True,
    download=True,
    transform=transforms.Compose(
        [transforms.ToTensor()])
)
```

- The dataset is filtered to only have digits labeled 0 and 1 (first `n_samples` of each). Then the `np.append` function is used to concatenate the indices of the two classes (0 and 1) and then filter the dataset using these indices. The `np.where` function returns the indices of elements in an array that satisfy a given condition. The `X_train.data` and `X_train.targets` attributes are then updated to include only the selected samples. The `X_train.data` attribute contains the image data, while `X_train.targets` contains the corresponding labels.

- The PyTorch DataLoader [**?**] is defined for the filtered training data, this is used to load the data in batches during training. The `shuffle=True` argument ensures that the data is shuffled before each epoch, which helps improve the model's capability to generalize.

```
train_loader = DataLoader(X_train,
batch_size=batch_size, shuffle=True)
```

- The MNIST dataset is loaded again, this time with `train=False` to indicate that we want to create the test set.
  The test dataset is filtered to only keep digits labeled 0 and 1 (as in the traning set).

- When creating the quantum neural network, a key decision must be made: **how to encode the classical input features into a quantum state** (a process known as *feature mapping* or *data encoding*). Qiskit provides several options for this, the most common being:
  - **ZFeatureMap [4]**
    A simple quantum feature map where each input value is encoded as a rotation on a single qubit using Rz gates. This approach does not include entanglement and is more fitting for simple datasets and models with lower computational complexity.
  - **ZZFeatureMap [5]**
    A more expressive feature map that includes both single-qubit rotations and entangling gates (ZZ interactions) between qubits. This allows the model to capture complex relationships between features, making it more suitable for richer or more structured datasets.
  - **PauliFeatureMap [6]** – a generalized feature map that supports different combinations of Pauli operators.

  These feature maps are not just about storing the features — they are essential for **embedding classical data (such as pixel values from images)** into the quantum circuit so that the quantum neural network can process it.

  The choice of feature map directly influences the expressiveness and performance of the quantum model. In this tutorial, the chosen feature map is the ZZFeatureMap.

- The next choice is the **ansatz** (the quantum circuit structure). The ansatz defines the architecture of the quantum neural network, including the number of qubits, layers, and gates used. Qiskit provides several predefined ansatz classes, such as:
  - **EfficientSU2 [7]**
    A general purpose ansatz that uses a combination of single-qubit rotations and entangling gates. It is designed to be efficient in terms of circuit depth and is suitable for a wide range of problems.
  - **RealAmplitudes [8]**
    This ansatz uses real-valued parameters and is often used for variational algorithms. It can be more expressive than EfficientSU2 but may require more qubits and gates.
  - **TwoLocal [9]**
    A flexible ansatz that allows users to specify the types of gates and the connectivity between qubits. It can be tailored to specific problems but may require more tuning.

  The choice of ansatz is crucial as it determines the expressiveness and complexity of the quantum model. In this tutorial the ansatz used is the RealAmplitudes.

```
def create_qnn():
    feature_map = ZZFeatureMap(2)
    ansatz = RealAmplitudes(
        2, reps=1)
    qc = QuantumCircuit(2)
    qc.compose(
        feature_map, inplace=True)
    qc.compose(
        ansatz, inplace=True)

    qnn = EstimatorQNN(
        circuit=qc,
        input_params=
        feature_map.parameters,
        weight_params=
        ansatz.parameters,
        input_gradients=True,
        estimator=estimator,
    )
    return qnn

qnn4 = create_qnn()
```

- The next step is to define the neural network that includes a quantum layer.

  The class Net [10] is a custom neural network built by using PyTorch. The network created in the tutorial combines classical layers (such as convolutional and linear layers) with a quantum neural network (QNN) layer created earlier with Qiskit. The QNN is integrated using the TorchConnector [11], which allows it to be used just like any other PyTorch layer and supports hybrid quantum-classical training via backpropagation.

  There are two convolutional layers, typically used for image data. $Conv2d(1, 2, kernel_size = 5)$ means the first layer takes 1 input channel grayscale image and outputs 2 feature maps using 5x5 filters. The second layer takes 2 channels and produces 16 feature maps.

Dropout2d [12] is a regularization layer that randomly zeros out entire channels during training to prevent overfitting.

fc1 and fc2 are fully connected (linear) layers. The first one takes the flattened output of the convolutional layers (256 features) and reduces it to 64 features. The second one takes the 64 features and outputs 2 features, which are then passed to the QNN.

The Qiskit quantum neural network is then wrapped using the TorchConnector, allowing it to be used as a layer in the PyTorch model. The QNN takes the 2-dimensional input from fc2 and produces a 1-dimensional output. The parameters (weights) of the quantum circuit are initialized randomly in the interval $[-1, 1]$. Finally, fc3 is another fully connected layer that takes the 1-dimensional output from the QNN and produces a final output of 2 features (which can be interpreted as probabilities for binary classification).

```
class Net(Module):
def __init__(self, qnn):
    super().__init__()
    self.conv1 =
    Conv2d(1, 2, kernel_size=5)
    self.conv2 =
    Conv2d(2, 16, kernel_size=5)
    self.dropout =
    Dropout2d()
    self.fc1 =
    Linear(256, 64)
    self.fc2 =
    Linear(64, 2)
    self.qnn
    = TorchConnector(qnn)
    self.fc3 = Linear(1, 1)
```

- The next step is to define the Forward pass. It applies the first convolutional layer with a ReLU [13] activation, followed by 2x2 max pooling (downsampling). The same steps are repeated for the second convolutional layer. After feature extraction with convolutional layers and dropout for regularization, the data is flattened and passed through fc1 and fc2. As mentioned before, the output, a 2D vector, is then processed by the quantum neural network (QNN) layer. Its result goes through another linear layer (fc3), and the final output is a concatenation of the prediction and its complement, representing probabilities for binary classification (digit 0 vs 1).

```
def forward(self, x):
    x = F.relu(self.conv1(x))
    x = F.max_pool2d(x, 2)
    x = F.relu(self.conv2(x))
    x = F.max_pool2d(x, 2)
    x = self.dropout(x)
    x = x.view(x.shape[0], -1)
    x = F.relu(self.fc1(x))
    x = self.fc2(x)
    x = self.qnn(x)  # apply QNN
    x = self.fc3(x)
    return cat((x, 1 - x), -1)
```

- Finally it's essencial to inicialize the model, optimizer and loss function.

  The optimizer is responsible for updating the model's parameters during training. In this case, the Adam optimizer [14] is used with a learning rate of 0.01. Adam is a first-order gradient-based optimizer that combines momentum, which uses past gradients, and adaptive learning rates that scale the step size based on how frequently a parameter is updated. It is commonly used for deep learning tasks due to its speed and robustness. One of its key advantages is the adaptive learning rate per parameter, which often leads to good convergence. Other optimizers are explored in the second tutorial.

  The loss function is used to measure the difference between the predicted output and the true labels. In this case, the $NegativeLogLikelihoodLoss$ [15] is used. The model is trained for 10 epochs. In each epoch, it iterates over the training batches and performs the following steps:
  - Gradients are reset with `optimizer.zero_grad()`.
  - A forward pass is performed to get the model's predictions.
  - The loss between predictions and true labels is computed.
  - A backward pass computes gradients.
  - The model updates its weights using the optimizer.
  - Loss values for the epoch are stored and averaged.
- A new quantum neural network is created and the new trained weights are loaded.
  The model is then evaluated on the test dataset. First, it is set to evaluation mode using `model.eval()`. Gradient computation is disabled using `no_grad()` to speed up inference and save memory.

  For each batch in the test dataset, the following steps are performed:
  - The model outputs a prediction for the input data;
  - If the output has an unexpected shape, it is reshaped appropriately;
  - The predicted label is obtained using `argmax` [16];
  - The number of correct predictions is counted by comparing with the true labels;
  - The loss is computed using the same loss

function as in training.

At the end, the average loss and accuracy are printed for each epoch. The accuracy is calculated as the ratio of correct predictions to the total number of samples in the test dataset.

Second tutorial

In contrast, the second tutorial "The Quantum Convolution Neural Network" [17] adopts a fully quantum model with custom built quantum convolutional and pooling layers. The tutorial expands the concepts of pooling layers, creating the pooling and convolution circuits and a custom dataset.

- The first step in this tutorial is to define a two bit convolutional circuit wich is used to perform a convolution operation on two qubits. This is done by applying a series of quantum gates to the qubits, including rotations and controlled-X (CX) gates, to create a quantum state that represents the convolution operation. The parameters of the circuit are defined as a vector of parameters, which can be optimized during training.

```
def conv_circuit(params):
    target = QuantumCircuit(2)
    target.rz(-np.pi / 2, 1)
    target.cx(1, 0)
    target.rz(params[0], 0)
    target.ry(params[1], 1)
    target.cx(0, 1)
    target.ry(params[2], 1)
    target.cx(1, 0)
    target.rz(np.pi / 2, 0)
    return target
params = ParameterVector("\theta",
 length=3)
circuit = conv_circuit(params)
```

- A convolutional layer is then constructed by composing multiple convolutional circuits. The result is a quantum circuit that can be applied to the qubits. This layer is used to perform convolution operations on the input data, extracting features from the quantum state.

- A pooling circuit is defined (`pool_circuit`). This circuit is used to perform a pooling operation on two qubits, which is another fundamental operation in quantum convolutional neural networks. The pooling operation reduces the dimensionality of the data while retaining important features, wich is similar to classical pooling operations in classical convolutional neural networks.

```
def pool_circuit(params):
    target = QuantumCircuit(2)
    target.rz(-np.pi / 2, 1)
    target.cx(1, 0)
    target.rz(params[0], 0)
    target.ry(params[1], 1)
    target.cx(0, 1)
    target.ry(params[2], 1)
    return target
```

- The pooling layer is created by composing several pooling circuits, `pool_layer` function.

- A synthetic dataset of images is created to train the QCNN. Each image is either a horizontal or vertical line. The images are generated using the `generate_dataset` function.

- The dataset is split into training and testing sets. The classifier is then trained using a quantum neural network with the defined convolutional and pooling layers.

```
images, labels = generate_dataset(10)
train_images, test_images
, train_labels
, test_labels = train_test_split(
    images
, labels, test_size=0.3
, random_state=246
)
```

- The callback function is used to track the objective function value during training. It updates the plot of the objective function value against the iteration number.

- The ZfeatureMap was defined as the feature map, with 8 qubits. The ansatz circuit is created with 8 qubits aswell. The convolutional and pooling layers are defined using the `conv_layer` and `pool_layer` functions. These layers are composed into the ansatz circuit, which are then used in the quantum neural network. The convolutional layers are applied first, followed by the pooling layers (3 of each) wich as mentioned before, reduce the dimensionality of the data while retaining important features. The final ansatz circuit is a combination of these layers. Then, the ansatz circuit is combined with the feature map.

- The observable is defined as a SparsePauliOp [18] with the operator ("Z" + "I" * 7), which will measure the Z component of the state on the first qubit while leaving the others unchanged. This observable is used to extract information from the quantum state after the ansatz has been applied. The circuit is decomposed to avoid data duplication when passed to the quantum neural network (QNN). This ensures the circuit is executed efficiently in the EstimatorQNN [19] model.

```
circuit = QuantumCircuit(8)
circuit.compose(feature_map
, range(8), inplace=True)
circuit.compose(ansatz
, range(8), inplace=True)
```

```
observable =
SparsePauliOp.from_list([
    ("Z" + "I" * 7, 1)])

qnn = EstimatorQNN(
    circuit=circuit.decompose(),
    observables=observable,
    input_params=feature_map.parameters,
    weight_params=ansatz.parameters,
    estimator=estimator,
)
```

- The classifier is then initialized and trained using the training data. The NeuralNetworkClassifier is initialized with the QNN and the COBYLA optimizer [20] (with a maximum of 200 iterations). This optimer differs from the Adam optimizer used in the first tutorial.

  Differenced betweenn Adam and COBYLA:

  - **Adam Optimizer**:
    - **Momentum**: Tracks the exponentially decaying average of past gradients to help avoid oscillations and speed up convergence.
    - **Adaptive Learning Rates**: Adjusts the learning rates for each parameter based on past gradients, improving convergence rates for different parameters in neural networks.
    - **Bias Correction**: Corrects the initial bias in the moment estimates, especially for small t (iteration steps).
  - **Key Hyperparameters of Adam**:
    - `alpha`: Learning rate.
    - `beta1`: Exponential decay rate for the first moment estimate.
    - `beta2`: Exponential decay rate for the second moment estimate.
    - `epsilon`: Small constant to prevent division by zero.
  - **COBYLA Optimizer**:
    - **Derivative-Free**: COBYLA is a derivative-free optimization method suitable for problems without gradients or where gradient calculation is expensive.

  A callback function (callback_graph) is then passed to track the objective function during training. The classifier is trained using the training data (x,y).

  During textttclassifier.fit(), the following occurs:

  - The NeuralNetworkClassifier performs a forward pass by computing the output for each input image x in `train_images`.
  - The data is passed through the quantum circuit defined by `feature_map` and `ansatz`, and the output is compared to the true labels y.
  - The QNN is a parameterized model, and this forward pass generates predictions by applying the quantum operations defined in the ansatz to the data encoded in the quantum feature map.

  Then, when doing the backward pass, the gradients of the loss function are calculated the with respect to the parameters (weights) of the network and those parameters are updated in order to minimize the loss. This is where the optimization process happens.

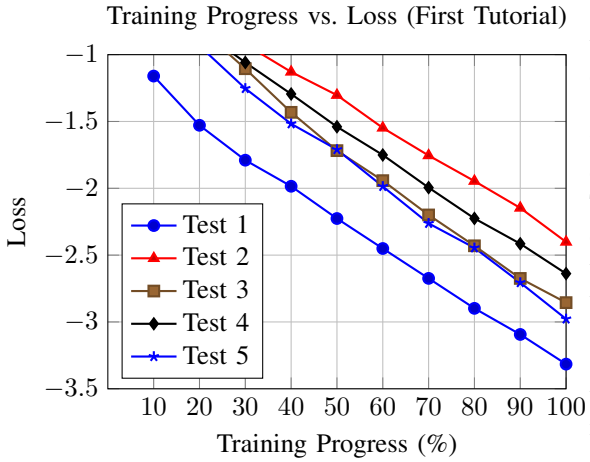
```
initial_point =
 np.random.random(qnn.num_weights)
classifier = NeuralNetworkClassifier(
    qnn,
    optimizer=COBYLA(maxiter=200),
    callback=callback_graph,
    initial_point=initial_point,
)
x = np.asarray(train_images)
y = np.asarray(train_labels)
objective_func_vals = []
classifier.fit(x, y)
y_predict = classifier.predict(
    test_images)
x = np.asarray(test_images)
y = np.asarray(test_labels)
```

Diferent types of HCQNNs

The first tutorial focuses on a direct application of a quantum neural network in a hybrid classical-quantum neural network model, using a known dataset like MNIST. The model begins with classical processing (convolutions, pooling, fully connected layers), passes through quantum circuits for feature extraction, and concludes with a classical layer for classification.
In contrast, the second tutorial adopts a more experimental approach, where data is processed classically and then converted into a quantum format. The QCNN uses convolutional and pooling layers to extract features, and the NeuralNetworkClassifier trains the model. Precision is evaluated, and results are displayed in a classical manner.

STATISTICS AND RESULTS OF THE TUTORIAL

First tutorial



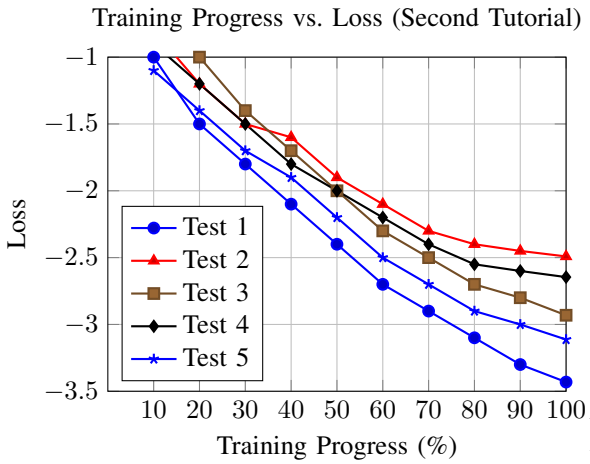
	Test 1	Test 2	Test 3	Test 4	Test 5
Final loss	-3.5102	-2.5000	-2.9971	-2.6358	-3.4921
accuracy	100%	99%	100%	100%	100%

TABLE I: accuracy and final loss of the tests

These tests were simulated locally, with a manual seed differing for each test. The accuracy was based on 50 training samples and 50 test samples, with the model trained for 10 epochs (one full pass through the entire training dataset). The final loss was calculated after the training process.

The results show that the model achieved high accuracy across all tests, with the final loss values indicating good convergence. The training progress was visualized, showing a steady decrease in loss as the training progressed. While all runs achieved high accuracy ($\geq$99%), the final loss varied, indicating that the model's performance can be sensitive to the initial conditions and the specific training data used.

Second tutorial and differences



The model was trained using the COBYLA optimizer for a maximum of 200 iterations. The results show

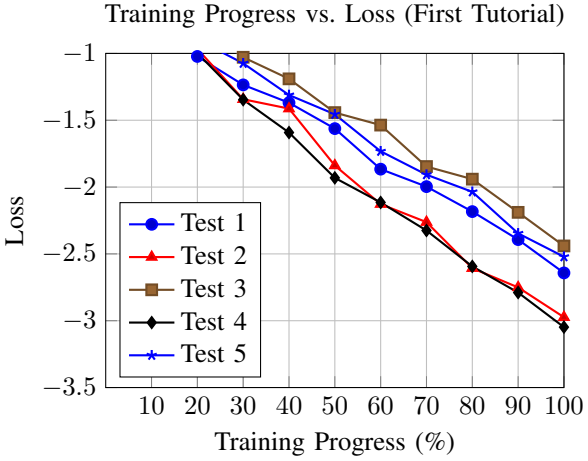
	Test 1	Test 2	Test 3	Test 4	Test 5
Final loss	-3.4312	-2.4912	-2.9313	-2.6458	-3.1129
accuracy	100%	66.67%	100%	100%	33.33%

TABLE II: accuracy and final loss of the tests

that the model achieved varying accuracy across different tests, with some tests achieving 100% accuracy while others had lower performance. The final loss values indicate that the model converged to different levels of performance depending on the test. The training progress was visualized, showing a decrease in loss as the training progressed, although the convergence was not as smooth as in the first tutorial.

The average performance disparity between the two tutorials stems from several architectural and implementation differences. The second tutorial adopts a fully quantum-centric approach that lacks any form of classical preprocessing or feature extraction, making it harder for the quantum model to generalize from high-dimensional input data. The usage of zfeature map (although with more qubits) in the second tutorial also limits the model's ability to capture complex patterns compared to the first tutorial's classical CNN layers. Additionally, the second tutorial's reliance on a derivative-free optimizer (COBYLA) contrasts with the first tutorial's Adam optimizer, which is more suited for gradient-based learning in hybrid models. These factors contribute to the observed differences in performance and accuracy between the two tutorials.

The first tutorial also found success with the artifical dataset created for the second tutorial:

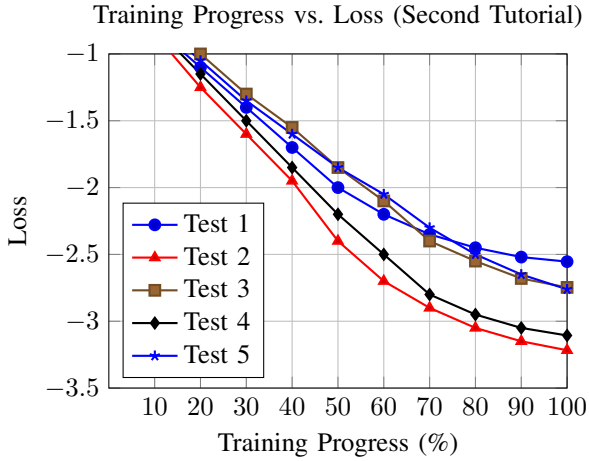


	Test 1	Test 2	Test 3	Test 4	Test 5
Final loss	-2.5541	-3.2172	-2.7470	-3.1065	-2.7612
accuracy	99%	100%	100%	100%	100%

TABLE III: accuracy and final loss of the tests

While the second tutorial, when trying to classify the MNIST dataset used in the first one, achieved around 50% accuracy with very little loss convergence. This again may be to the way the second tutorial treats the

data in a more quantum-centric manner, meaning that the images need to be resized to fit the quantum model, losing its features and becoming too noisy.



	Test 1	Test 2	Test 3	Test 4	Test 5
Final loss	-2.5012	-3.2629	-2.7570	-3.1023	-2.5610
Accuracy	43%	75%	63%	62%	54%

TABLE IV: Accuracy and final loss of the tests

WHAT WORKS BEST AND CREATING A CUSTOM HCQNN

Based on the results obtained from the first two tutorials, a custom hybrid quantum convolutional neural network that combines the best features of both approaches was developed.
Like in the first tutorial, the Estimator() and MNIST dataset are loaded. After loading the MNIST training set, the transformation to convert images to tensors is applied. 50 samples are loaded for each of digits 0 and 1 from the test set.

Trying to manually process the images using NEQR and FRQI

Flexible Representation of Quantum Images (FRQI) is a method encodes both the color (grayscale intensity) and position of each pixel into a quantum state. Specifically, the intensity information is represented by rotating a qubit to a specific angle, while the pixel positions are mapped using basis states.

Novel Enhanced Quantum Representation (NEQR), unlike FRQI, uses multiple qubits to represent the grayscale value of each pixel directly, allowing for a more precise encoding of image information.

The NEQR and FRQI methods were tested in the HCQNN [21] [22], instead of using ZZFeatureMap or ZfeatureMap. The results of using NEQR and FRQI instead of quantum feature maps were not as good as expected. These methods are more complex and require careful tuning of parameters, which can lead to longer training times and less intuitive results. More experimental strategies were studied, such as QPIE [23] wich is a more structure-aware encoding inspired by polar coordinates and DCT-EFRQI [24], that efficiently takes advantage of classical Discrete Cosine Transform. This would maybe yield slightly better results but similar results to the other classical encoding methods mentioned.
Because of this, the decision came down to using the ZFeatureMap or ZZFeatureMap.

Deciding between zfeaturemap and zzfeaturemap

Like mentioned before, the ZFeatureMap is a simpler feature map that encodes classical data into quantum states using single-qubit rotations. It is computationally less expensive and easier to implement, making it suitable for smaller datasets or simpler models. However, it may not capture complex relationships between features as effectively as the ZZFeatureMap. The ZZFeatureMap, on the other hand, includes both single-qubit rotations and entangling gates (ZZ interactions) between qubits. This allows the model to capture complex relationships between features, making it more suitable for richer or more structured datasets. However, it is computationally more expensive, but in this case, the dataset is small enough that the computational cost is not a significant issue. Because of this, the ZZFeatureMap was chosen for the model.

Deciding between Adam and COBYLA

In this hybrid quantum-classical neural network setup, using the Adam optimizer is more appropriate than COBYLA. Like in the first tutorial, the model integrates both classical layers (such as convolutional and fully connected layers) and a quantum neural network wrapped with TorchConnector, which enables automatic differentiation through PyTorch's autograd system. This setup relies on gradient-based optimization, where the `loss.backward()` call computes gradients, and `optimizer.step()` updates the parameters accordingly. In contrast, COBYLA is a gradient-free optimizer from SciPy [25] that does not integrate with PyTorch's autograd and cannot update parameters through backpropagation. Using COBYLA in this context would prevent the model from training properly, as it cannot optimize the classical network components or interface with PyTorch's training loop. Therefore, for training hybrid models end-to-end within PyTorch, gradient-based optimizers like Adam should be used to ensure all components are updated correctly.

Deciding the ansatz

The ansatz is a crucial component of quantum neural networks, as it defines the structure and complexity

of the quantum circuit used for feature extraction. The choice of ansatz can significantly impact the performance of the model.
The chosen ansatz, from the ones listed in the first tutorial, was the TwoLocal ansatz. This ansatz is flexible and allows users to specify the types of gates and the connectivity between qubits. It can be tailored to specific problems, making it suitable for a wide range of applications.
The parameters chosen were:

```
ansatz = TwoLocal(num_qubits=2,
rotation_blocks='ry',
entanglement_blocks='cz', reps=1)
```

The output layer

nn.sequential [26] was used to create the output layer, which is a simple way to stack layers in PyTorch. The output layer consists of a linear layer followed by a softmax [27] activation function. The linear layer maps the output of the quantum neural network to the desired number of classes (in this case, 2 for binary classification). Then, the softmax activation function converts the raw output scores into probabilities, making it also suitable for multi-class classification tasks. This is better that the previous approach, where the output was a concatenation of the prediction and its complement. The new approach is more straightforward and aligns better with standard practices in neural network design.

Adding evolutionary operators

To complement the gradient-based optimization performed by the Adam optimizer, evolutionary-inspired strategies were also applied to the model parameters. This hybrid optimization approach introduces stochastic variation through crossover and mutation operators, inspired by genetic algorithms. These operators are applied at the end of each training epoch to promote exploration of the parameter space and potentially escape local minima.

The crossover method used is a single-point crossover, commonly applied in evolutionary algorithms to combine traits from two parent solutions. It works by first flattening both parameter tensors into one-dimensional vectors. A random crossover point is then selected somewhere between the first and last index. At this point, the vectors exchange their tail segments, producing two new offspring vectors that blend characteristics of both parents. These new vectors are subsequently reshaped back into the original tensor dimensions to maintain compatibility with the model. This process enables the model to explore new combinations of parameters, encouraging diversity in the population while preserving useful traits from the previous generation.

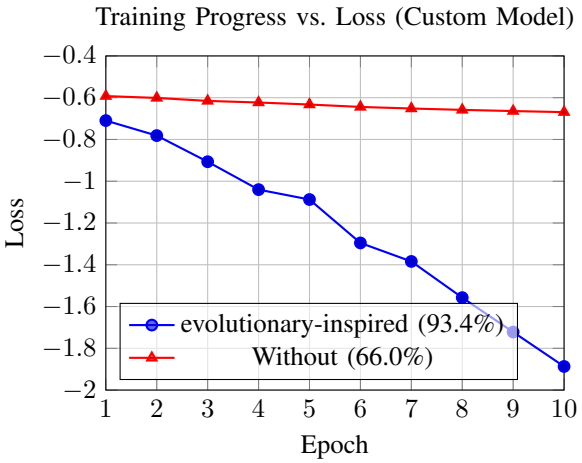
```
def crossover(param1, param2):
    flat1 = param1.flatten()
    flat2 = param2.flatten()
    length = flat1.size(0)
    point = torch.randint(1, length,
     (1,)).item()
    new1 = torch.cat([flat1[:point],
     flat2[point:]])
    new2 = torch.cat([flat2[:point],
     flat1[point:]])
    return new1.view(param1.shape),
     new2.view(param2.shape)
```

The mutation method employed is a Gaussian mutation with per-element mutation probability, a standard approach for introducing variability into model parameters. This technique begins by generating a mask tensor of the same shape as the parameter tensor, where each element has a fixed probability (e.g., 5%) of being selected for mutation. For every element in the tensor, Gaussian noise with zero mean and a specified standard deviation (mutation strength, typically 0.1) is generated. This noise is then selectively added only to those elements flagged by the mask. As a result, a subset of the parameter values is perturbed by small, random amounts, introducing stochastic changes that help the model explore new solutions and avoid premature convergence.

```
def mutation(param, mutation_rate=0.05,
 mutation_strength=0.1):
    mask = (torch.rand_like(param) <
     mutation_rate).float()
    noise = torch.randn_like(param) *
     mutation_strength
    param.data.add_(mask * noise)
```

Together, these evolutionary operators create a hybrid learning paradigm, blending backpropagation with biologically inspired stochastic search. While this adds computational overhead, it can improve convergence and robustness in non-convex and high-dimensional optimization settings, especially when combined with possibly noisy quantum components.

```
def apply_evolution(model):
    params = list(model.parameters())
    n = len(params)
    for i in range(0, n - 1, 2):
        if params[i].shape ==
         params[i + 1].shape:
            new1, new2 = crossover(
                params[i], params[i + 1])
            params[i].data.copy_(new1)
            params[i + 1].data.copy_(new2)
        else:
            pass
    for i in range(n):
        mutation(params[i])
```



These results had the same seed, batch size and 500 testing samples with 100 training. The model was trained for 10 epochs, with the final loss calculated after the training process.
The results show that the model achieved high accuracy with evolutionary operators, while the model without them did not improve significantly. The training progress was visualized, showing a steady decrease in loss as the training progressed. The evolutionary operators significantly improved the model's performance, demonstrating the possible effectiveness of combining gradient-based optimization with stochastic search methods in hybrid quantum-classical neural networks.

Pooling and convolution layers

These were used like in the first tutorial, but for simplicity, only one of each was created. The HybridNet class was created to encapsulate the model architecture and the forward pass:

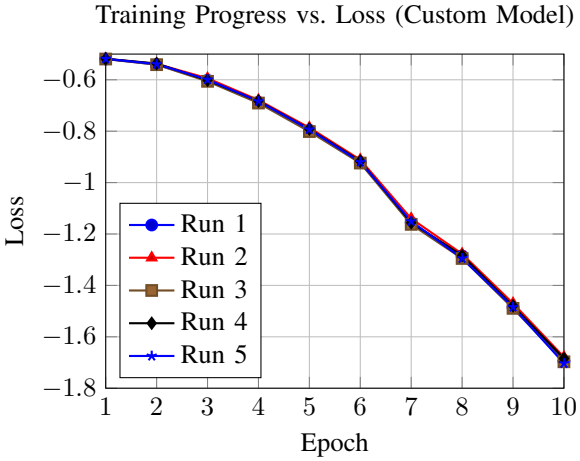
```
class HybridNet(Module):
    def __init__(self, qnn):
        super().__init__()
        self.conv1 = Conv2d(1, 4
        , kernel_size=5)
        self.pool =
        torch.nn.MaxPool2d(2)
        self.fc1 = Linear(
            4*12*12, 2)
         self.qnn =
        TorchConnector(qnn)
         self.fc2 =
        Linear(1, 1)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = self.pool(x)
        x = x.view(x.shape[0], -1)
        x = F.relu(self.fc1(x))
        x = self.qnn(x)
        x = self.fc2(x)
        return cat((x, 1 - x),
         dim=-1)
```

STATISTICS AND RESULTS OF THE CUSTOM MODEL

MNIST written digits dataset

Since the artifical dataset used in the second tutorial was too easy to classify, the binary classification of the MNIST dataset was used, with the digits 0 and 1. The same test and training size as in the first tutorial were as follows:



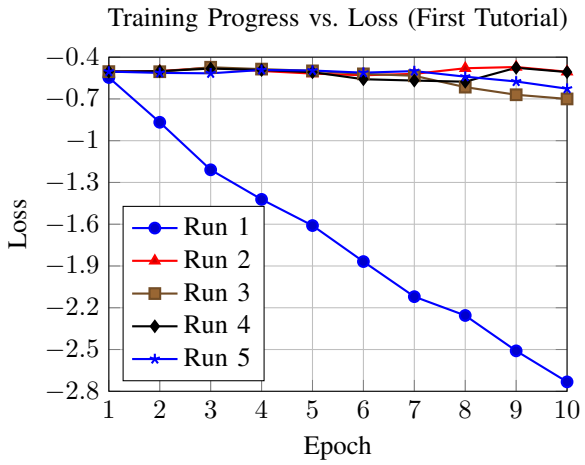
	Test 1	Test 2	Test 3	Test 4	Test 5
Final loss	-1.7407	-1.7141	-1.7451	1.7048	1.7465
accuracy	100%	99%	100%	100%	100%

TABLE V: accuracy and final loss of the tests

The results show that the model achieved high accuracy across all tests, with similar final loss values indicating good convergence. The training progress was visualized, showing a steady decrease in loss as the training progressed.

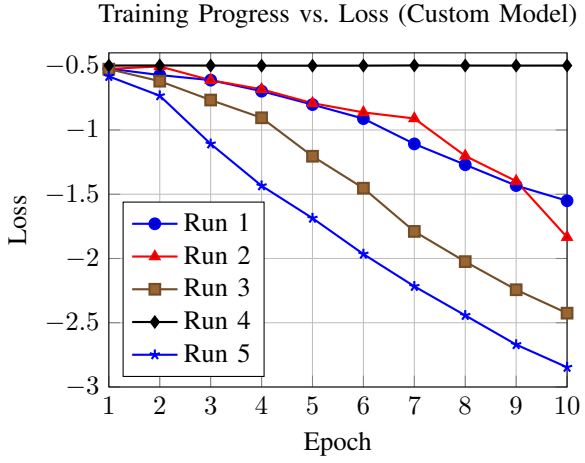
Fashion-MNIST dataset

These tests were performed using the fashion MNIST dataset [28] , which is more challenging than the MNIST dataset used in the first tutorial. The same training and testing size was used. A subset of classes (0 and 2, "T-shirt/top" and "Pullover") was selected to keep binary classification. The results obtained were as follows:



	Test 1	Test 2	Test 3	Test 4	Test 5
Final loss	-2.6044	-0.4413	-0.8352	-0.4623	-0.7545
accuracy	93%	47%	69%	49%	81%

TABLE VI: accuracy and final loss of the tests



	Test 1	Test 2	Test 3	Test 4	Test 5
Final loss	-1.3442	-1.7675	-2.2366	-0.4995	-2.6039
accuracy	91%	94%	93%	50%	93%

TABLE VII: accuracy and final loss of the tests

These results show that the custom HCQNN model performed well on the fashion MNIST dataset, achieving high accuracy across different tests. The final loss values indicate that the model converged to a good solution, with some tests achieving very low loss values. The training progress also shows a steady decrease in loss over the epochs, indicating that the model was learning effectively. The results, even with a small testing set, demonstrate the effectiveness of the built custom HCQNN, using the better features of the previous tutorials, while also testing different encoding methods and optimizers. These results were comparatively better and more stable (ignoring the outliers) than the results of the first tutorial in this dataset.

RUNNING CUSTOM HCQNN ON REAL HARDWARE

When running the custom HCQNN on real hardware, it is necessary to define the backend, which was defined by the least busy available with a minimum of 2 qubits. A pass manager and transpiler were also used in the quantum circuit to transform it using hardware-aware passes (gradients, shot schedules, and error mitigation are properly handled and no unsupported gate remains before runtime execution).

```
service = QiskitRuntimeService()
backend = service.least_busy(
    operational=True,
     simulator=False,
      min_num_qubits=2)
estimator = EstimatorV2(backend)

pass_manager =
 generate_preset_pass_manager(
    optimization_level=2,
     backend=backend)
qc = transpile(qc, backend)
```

Even with these optimizations, running the full HCQNN showed aboved depleted the full 10 minutes that come with the free IBM Quantum account. Very few forward passes (one job) were executed.

When the new IBM Cloud platform became available, a mini version of the HCQNN — with only 20 training samples and 10 testing samples, running for 5 epochs and with smaller convolutional and pooling sizes: `self.conv1 = Conv2d(1, 2, kernel_size=3), self.pool = torch.nn.MaxPool2d(2), self.fc1 = Linear(2 * 13 * 13, 2)` — was created. Even with the new `EstimatorV2` class [29] and these changes, the jobs didn't run to completion. By analyzing the jobs that were completed, it is possible to infer that the model was, in the final jobs, likely correctly inferring the classes of the images (handwritten digits). This was suggested by an `ev` result of approximately 0.99 ± 0.014, obtained in the primitive results from the last job. This indicates the QNN model confidently assigned a class close to 1, with a tight standard error (around 1.4%). Since this isn't sufficient to infer global model performance, future work could include a rule in the code that adds the true label of the image in the metadata, which could then be retrieved from completed jobs in a more efficient way. More runtime minutes would also resolve the root problem.

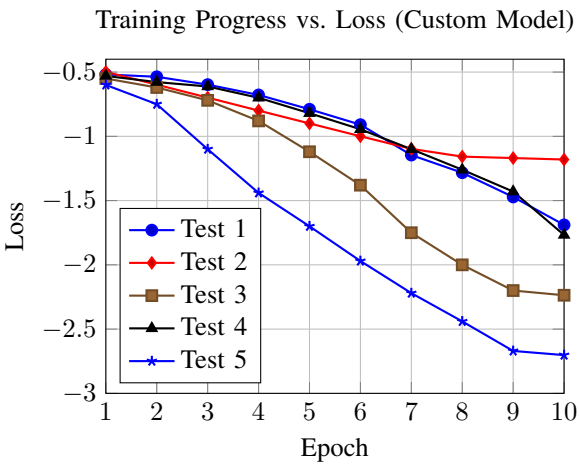
It is also possible to simulate the noise and errors of real hardware by using noise models, which can be created using the `NoiseModel` [30] class. The noise model can be created by adding quantum errors to

the gates used in the quantum circuit. The following code snippet shows how to create a noise model with depolarizing and thermal relaxation errors, which can be used to simulate the noise of real hardware:

```
noise_model = NoiseModel()
noise_model.add_all_qubit_quantum_error(
    depolarizing_error(0.01, 1),
     ['u1', 'u2', 'u3'])
noise_model.add_all_qubit_quantum_error(
    depolarizing_error(0.03, 2),
     ['cx'])
noise_model.add_all_qubit_quantum_error(
    thermal_relaxation_error(t1=50e3,
     t2=50e3, time=100,
      excited_state_population=0.),
    ['u2']
)

sim = AerSimulator(noise_model=
noise_model,shots=1024)

estimator = Estimator(options={
    'backend_options': {
        'noise_model': noise_model}})
```



	Test 1	Test 2	Test 3	Test 4	Test 5
Final loss	-1.6892	-0.4995	-2.2366	-1.7675	-2.7023
Accuracy	85%	42%	91%	95%	79%

TABLE VIII: Accuracy and final loss of the tests

The results show that the model still achieved high accuracy across all tests, with similar final loss values indicating good convergence. The training progress was visualized, showing a steady decrease in loss as the training progressed. The results obtained from the simulated noise model were similar to the noise-free model. The final loss indicates that the model was, in average and ignoring the outliers, able to learn effectively even with the noise and errors introduced by the noise model. The accuracy is lower and less consistent than the runs in the noise-free model. These tests were performed with random seeds and the fashion-MNIST dataset, with the same training and testing size as in the previowere similarus tests.

CONCLUSION

In this report, the implementation of hybrid quantum convolutional neural networks (HCQNNs) using Qiskit and PyTorch was explored by understanding the basics of quantum neural networks and their applications in image classification tasks. Through hands-on tutorials, it was learned how to construct QNNs, integrate them with classical layers, and train them on datasets like MNIST. Advanced concepts such as feature mapping, ansatz design, and the use of different optimizers were explored. Finally, the comparison was made of various QNN architectures and a custom-built HCQNN that leverages the strengths of both tutorials at using classical and quantum computing for good performance in image classification tasks. The results obtained from the experiments demonstrated the potential of HCQNNs in achieving competitive accuracy on image classification tasks. The combination of classical and quantum layers allows for efficient feature extraction and representation learning, making HCQNNs a promising avenue for future research in quantum machine learning.

FUTURE WORK

HQNN's for binary classification tasks were the ones explored in the tutorials, so when doing the custom HCQNN, the focus was on binary classification tasks. However, future work could involve extending the HCQNN to multi-class classification tasks, where the model can learn to classify images into multiple categories (using the MNIST datasets for example). This would require modifications to the output layer and loss function to accommodate multi-class outputs.

Also, as mentioned before, the NEQR and FRQI methods tested were not as effective as expected, so future work could involve exploring more advanced encoding techniques, such as QPIE or DCT-EFRQI, to improve the performance of the HCQNN on more complex datasets.

Running the HCQNN on real quantum hardware was not successful due to time constraints and the limitations of the free IBM Quantum account. Future work could involve optimizing the HCQNN for better performance on real hardware, such as reducing the number of qubits used or simplifying the model architecture to fit within the available resources. Additionally, exploring different quantum backends and hardware configurations could lead to improved results.

Finally, further research could be conducted on the integration of evolutionary-inspired optimization techniques with HCQNNs, as this approach showed promising results in improving model performance. Exploring different evolutionary operators and their

impact on training dynamics could lead to more robust and efficient HCQNNs. Comparing the performance of HCQNNs with these features and classical neural networks on various datasets could also provide insights into the advantages and limitations of quantum machine learning approaches.

REFERENCES

[1] Qiskit Community, "Torch connector — qiskit machine learning tutorials," https://github.com/qiskit-community/qiskit-machine-learning/blob/stable/0.8/docs/tutorials/05_torch_connector.ipynb, 2024.

[2] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019, pp. 8024–8035. [Online]. Available: https://arxiv.org/abs/1912.01703

[3] Y. LeCun and C. Cortes, "MNIST handwritten digit database," http://yann.lecun.com/exdb/mnist/, 2010.

[4] Qiskit Contributors, "Zfeaturemap class — qiskit documentation," https://docs.quantum.ibm.com/api/qiskit/qiskit.circuit.library.ZFeatureMap, 2025.

[5] ——, "Zzfeaturemap class — qiskit documentation," https://docs.quantum.ibm.com/api/qiskit/qiskit.circuit.library.ZZFeatureMap, 2025.

[6] ——, "Paulifeaturemap class — qiskit documentation," https://docs.quantum.ibm.com/api/qiskit/qiskit.circuit.library.PauliFeatureMap, 2025.

[7] ——, "Efficientsu2 ansatz — qiskit documentation," https://docs.quantum.ibm.com/api/qiskit/qiskit.circuit.library.EfficientSU2, 2025.

[8] ——, "realamplitudes ansatz — qiskit documentation," https://docs.quantum.ibm.com/api/qiskit/qiskit.circuit.library.RealAmplitudes, 2025.

[9] ——, "Twolocal ansatz — qiskit documentation," https://docs.quantum.ibm.com/api/qiskit/qiskit.circuit.library.TwoLocal, 2025.

[10] PyTorch Contributors, "Neural networks — pytorch documentation," https://docs.pytorch.org/tutorials/beginner/blitz/neural_networks_tutorial.html, 2025.

[11] Qiskit Contributors, "Torchconnector class — qiskit documentation," https://quantum.cloud.ibm.com/docs/en/api/qiskit-ibm-runtime/estimator, 2025.

[12] PyTorch Contributors, "Dropout2d — pytorch documentation," https://docs.pytorch.org/docs/stable/generated/torch.nn.Dropout2d.html, 2025.

[13] ——, "Relu — pytorch documentation," https://docs.pytorch.org/docs/stable/generated/torch.nn.ReLU.html, 2025.

[14] ——, "Adam — pytorch documentation," https://docs.pytorch.org/docs/stable/generated/torch.optim.Adam.html, 2025.

[15] ——, "Nllloss — pytorch documentation," https://docs.pytorch.org/docs/stable/generated/torch.nn.NLLLoss.html, 2025.

[16] ——, "Argmax — pytorch documentation," https://docs.pytorch.org/docs/main/generated/torch.argmax.html, 2025.

[17] Qiskit Community, "Quantum convolutional neural networks — qiskit machine learning tutorials," https://github.com/qiskit-community/qiskit-machine-learning/blob/stable/0.8/docs/tutorials/11_quantum_convolutional_neural_networks.ipynb, 2024.

[18] Qiskit Contributors, "Sparsepauliop — qiskit documentation," https://docs.quantum.ibm.com/api/qiskit/qiskit.quantum_info.SparsePauliOp, 2025.

[19] ——, "Estimatorqnn — qiskit documentation," https://qiskit-community.github.io/qiskit-machine-learning/stubs/qiskit_machine_learning.neural_networks.EstimatorQNN.html, 2025.

[20] ——, "Cobyla — qiskit documentation," https://docs.quantum.ibm.com/api/qiskit/0.37/qiskit.algorithms.optimizers.COBYLA, 2025.

[21] Q. Team, "How to start experimenting with quantum image processing," https://medium.com/qiskit/how-to-start-experimenting-with-quantum-image-processing-283dddcc6ba0, 2022.

[22] Qiskit Textbook Contributors, "Image processing with frqi and neqr — qiskit textbook," https://github.com/Qiskit/textbook/blob/main/notebooks/ch-applications/image-processing-frqi-neqr.ipynb, 2023.

[23] A. U. . T. D. Md Ershadul Haque *, Manoranjan Paul, "Advanced quantum image representation and compression using a dct-efrqi approach," *nature scientific reports*, 2023.

[24] P. S. B. V. P. Alok Anad, Meizhong Lyu, "Quantum image processing," 2022.

[25] SciPy Contributors, "scipy.optimize.minimize(method='cobyla') — scipy documentation," https://docs.scipy.org/doc/scipy/reference/optimize.minimize-cobyla.html, 2024.

[26] PyTorch Contributors, "Sequential — pytorch documentation," https://docs.pytorch.org/docs/stable/generated/torch.nn.Sequential.html, 2025.

[27] ——, "Softmax — pytorch documentation," https://docs.pytorch.org/docs/stable/generated/torch.nn.Softmax.html, 2025.

[28] ——, "Fashion-mnist dataset — pytorch documentation," https://github.com/zalandoresearch/fashion-mnist, 2025.

[29] Qiskit Contributors, "Estimator v2 — qiskit documentation," https://quantum.cloud.ibm.com/docs/en/api/qiskit-ibm-runtime/estimator-v2, 2025.

[30] ——, "Build noise models — qiskit documentation," https://quantum.cloud.ibm.com/docs/en/guides/build-noise-models, 2025.